

pyGC: A Python Package for Parametric and Non-Parametric Frequency-Domain Granger Causality Estimation

Vinicius Lima Cordeiro  ¹✉

¹ Your Institution, Country  Corresponding author

DOI: [10.21105/joss.XXXXX](https://doi.org/10.21105/joss.XXXXX)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [To be assigned](#) 

Reviewers:

- Pending

Submitted: DD MMM 2025

Published: Pending

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

pyGC is an open-source Python library for estimating Granger Causality (GC) in the frequency domain from multivariate time-series data. The package implements both the *parametric* pathway—fitting a Vector Auto-Regressive (VAR) model via the Yule-Walker equations and deriving the transfer function analytically—and the *non-parametric* pathway based on Wilson spectral factorization of a directly estimated cross-spectral matrix (Wilson, 1972; Dhamala et al., 2008). An optional covariance-based time-domain GC estimator and a JAX back-end that JIT-compiles the entire Wilson iteration loop via XLA are also provided. The library targets neuroscience applications (EEG, MEG, LFP) but is applicable to any domain where directional information flow between signals needs to be quantified.

Statement of Need

Granger Causality (Granger, 1969) is a standard tool for inferring directed connectivity from neural time series. Frequency-domain formulations (Geweke, 1982; Dhamala et al., 2008) are particularly popular because they reveal the frequency band at which the influence operates, a detail lost in time-domain summaries.

While individual routines exist in MATLAB toolboxes such as MVGC (Barnett & Seth, 2014) and in scattered Python snippets, a cohesive, tested, and pip-installable Python library that covers both estimation pathways, conditional GC, covariance-based GC, synthetic benchmark models, and GPU-accelerated computation has been lacking. pyGC fills this gap with a clean NumPy/SciPy API, a full pytest suite, and optional JAX acceleration.

Background

Granger Causality in the Frequency Domain

Given a stationary multivariate process $\mathbf{X}(t)$, its VAR representation of order m is

$$\mathbf{X}(t) = \sum_{k=1}^m \mathbf{A}_k \mathbf{X}(t-k) + \varepsilon(t), \quad \varepsilon(t) \sim \mathcal{N}(\mathbf{0}, \Sigma), \quad (1)$$

where $\mathbf{A}_k$ are the coefficient matrices and Σ is the noise covariance. The transfer function is $\mathbf{H}(f) = [\mathbf{I} - \sum_k \mathbf{A}_k^\top e^{-2\pi i k f / F_s}]^{-1}$ and the cross-spectral matrix is $\mathbf{S}(f) = \mathbf{H}(f) \Sigma \mathbf{H}^*(f)$.

For a bivariate system (X, Y) the pairwise spectral GC measures are (Geweke, 1982; Lima et

al., 2020):

$$I_{X \rightarrow Y}(f) = \ln \frac{S_{YY}(f)}{\tilde{H}_{YY}(f) \Sigma_{YY} \tilde{H}_{YY}^*(f)}, \quad (2)$$

$$I_{Y \rightarrow X}(f) = \ln \frac{S_{XX}(f)}{H_{XX}^\circ(f) \Sigma_{XX} H_{XX}^{\circ*}(f)}, \quad (3)$$

where $\tilde{H}_{YY} = H_{YY} + (\Sigma_{YX}/\Sigma_{XX}) H_{XY}$ and $H_{XX}^\circ = H_{XX} + (\Sigma_{XY}/\Sigma_{YY}) H_{YX}$ are the intrinsic transfer functions after absorbing off-diagonal noise correlations.

Wilson Spectral Factorization

When VAR model fitting is not desired, the transfer function and noise covariance can be recovered directly from a non-parametric estimate of $S(f)$ via the Wilson spectral factorization (Wilson, 1972). The algorithm iteratively finds an analytic function $\Psi(f)$ such that $S(f) = \Psi(f) \Psi^*(f)$, yielding $H(f)$ and Σ without assuming a finite-order VAR model.

Conditional and Covariance-Based GC

For multivariate ($p > 2$) systems, pyGC provides conditional GC (Geweke, 1984), which removes shared driving by conditioning on all remaining channels. Separate Wilson factorizations are run on each $(p-1)$ -dimensional reduced model; the results are assembled into a $p \times p$ GC matrix. These reduced-model factorizations are embarrassingly parallel and are executed with `joblib` when `n_jobs > 1`.

A covariance-based time-domain GC estimator (Vinck et al., 2015) is also included via `covgc_time`, which computes pairwise directional influences from sliding-window covariance matrices without an explicit spectral transform.

Package Structure

pyGC is organised as a single installable package (`pygc`) with the following modules:

- `parametric` — Yule-Walker VAR fitting (`YuleWalker`) and transfer-function computation (`compute_transfer_function`).
- `non_parametric` — Wilson spectral factorization (`wilson_factorization`).
- `granger` — bivariate GC (`granger_causality`), conditional time-domain GC (`conditional_granger_causality`), and conditional spectral GC (`conditional_spec_granger_causality`).
- `covgc` — covariance-based time-domain GC (`covgc_time`).
- `tools` — cross-correlation (`xcorr`), demeaning, and regularised determinant (`rdet`).
- `ar_model` — synthetic benchmark processes: the two-variable AR model of Dhamala et al. (2008) and the five-variable model of Baccalá & Sameshima (2001).
- `spectral_analysis` — spectral estimation helpers (Fourier CSD, Morlet wavelets, Welch power spectrum) built on MNE-Python (Gramfort et al., 2013).
- `_jax_backend` — optional JAX/XLA back-end with a JIT-compiled Wilson loop for CPU/GPU acceleration.

Implementation Details

Vectorised Wilson Factorization

The reference implementation of Wilson factorization iterates over frequency bins in Python, which is prohibitively slow for large spectral matrices or high-resolution frequency axes. pyGC replaces every inner loop with batch NumPy operations: `np.linalg.inv` is applied to the full $(2N \times m \times m)$ tensor in a single call, and the Plus operator is implemented via `np.fft.fft/iff`

along the frequency axis. This yields a roughly $16\times$ wall-clock speedup over the loop-based reference on a modern CPU.

JAX Back-End

When JAX (Bradbury et al., 2018) is installed, `wilson_factorization_jax` exposes a JIT-compiled version of the entire Wilson loop via `jax.lax.while_loop`. The convergence condition is encoded as a JAX boolean predicate so that the compiled kernel exits as soon as the matrix 1-norm drops below `tol` without returning to Python between iterations. CPU and GPU execution are both supported and are selected automatically by the JAX device back-end.

Parallelism

Conditional (spectral) GC requires one Wilson factorization per channel (reduced models). These are independent and are dispatched via `joblib.Parallel` with a thread pool (`prefer='threads'`), which avoids serialisation overhead for NumPy-heavy workloads.

Usage Example

The following snippet demonstrates the bivariate parametric workflow using the built-in Dhamala benchmark model, where channel Y drives channel X at 40 Hz:

```
import numpy as np
from pygc import ar_model, YuleWalker
from pygc import compute_transfer_function, granger_causality
from pygc.spectral_analysis import compute_freq

Fs = 200
data = ar_model.ar_model_dhamala(N=5000, Trials=20, Fs=Fs, C=0.25)

f = compute_freq(data.shape[2], Fs)
m = 2
AR = np.zeros((m, 2, 2))
SIG = np.zeros((2, 2))
for trial in range(data.shape[1]):
    a, s = YuleWalker(data[:, trial, :], m)
    AR += a / data.shape[1]
    SIG += s / data.shape[1]

H, S = compute_transfer_function(AR, SIG, f, Fs)
Ix2y, Iy2x, Ixy = granger_causality(S, H, SIG)
# Iy2x peaks at 40 Hz; Ix2y is near zero.
```

An identical result can be obtained non-parametrically by replacing the Yule-Walker step with CSD estimation followed by `wilson_factorization`.

Testing

pyGC ships with a pytest suite of 43 tests covering:

- Correctness of VAR fitting (Yule-Walker residuals).
- Spectral factorization convergence and reconstruction error.
- GC direction recovery on the Dhamala and Baccalá benchmark models ($I_{Y \rightarrow X} > I_{X \rightarrow Y}$).
- Conditional GC matrix sparsity on the Baccalá five-variable model.
- Numerical stability of `covgc_time` for various window and lag parameters.

- API consistency between the NumPy and JAX back-ends when JAX is available.

Tests are run with pytest and a coverage report is generated via pytest-cov.

Related Software

MVGC (Barnett & Seth, 2014) is a comprehensive MATLAB toolbox for GC analysis but does not provide a Python interface. **MNE-Connectivity** provides spectral connectivity measures in Python but does not implement Wilson factorization or conditional spectral GC. **nitime** offers VAR-based GC but has not been actively maintained. pyGC complements these tools by providing a modern, tested, pip-installable Python library that covers the full non-parametric pipeline with GPU acceleration.

Acknowledgements

The theoretical foundations of this package were developed alongside the tutorial paper Lima et al. (2020). The Wilson factorization implementation is based on the algorithm described in Dhamala et al. (2008).

Granger, C. W. J. (1969). Investigating causal relations by econometric models and cross-spectral methods. *Econometrica*, 37(3), 424–438. <https://doi.org/10.2307/1912791>

Geweke, J. (1982). Measurement of linear dependence and feedback between multiple time series. *Journal of the American Statistical Association*, 77(378), 304–313. <https://doi.org/10.2307/2287243>

Geweke, J. (1984). Measures of conditional linear dependence and feedback between time series. *Journal of the American Statistical Association*, 79(388), 907–915. <https://doi.org/10.2307/2288723>

Wilson, G. T. (1972). The factorization of matricial spectral densities. *SIAM Journal on Applied Mathematics*, 23(4), 420–426. <https://doi.org/10.1137/0123044>

Dhamala, M., Rangarajan, G., & Ding, M. (2008). Estimating Granger causality from Fourier and wavelet transforms of time series data. *Physical Review Letters*, 100(1), 018701. <https://doi.org/10.1103/PhysRevLett.100.018701>

Baccalá, L. A., & Sameshima, K. (2001). Partial directed coherence: a new concept in neural structure determination. *Biological Cybernetics*, 84(6), 463–474. <https://doi.org/10.1007/PL00007990>

Barnett, L., & Seth, A. K. (2014). The MVGC multivariate Granger causality toolbox: A new approach to Granger-causal inference. *Journal of Neuroscience Methods*, 223, 50–68. <https://doi.org/10.1016/j.jneumeth.2013.10.018>

Lima, V., et al. (2020). Granger causality in the frequency domain: derivation and applications. *Revista Brasileira de Ensino de Física*, 42, e20190105. <https://doi.org/10.1590/1806-9126-rbef-2019-0105>

Vinck, M., et al. (2015). How to detect the Granger-causal flow direction in the presence of additive noise? *NeuroImage*, 108, 301–318. <https://doi.org/10.1016/j.neuroimage.2014.12.017>

Gramfort, A., et al. (2013). MEG and EEG data analysis with MNE-Python. *Frontiers in Neuroscience*, 7, 267. <https://doi.org/10.3389/fnins.2013.00267>

Bradbury, J., et al. (2018). JAX: composable transformations of Python+NumPy programs. <http://github.com/google/jax>